Lab Purpose

The goal of this lab is to configure 3 fully secured SSIDs in a corporate environment. Two SSIDs will be authenticated using WPA2 and One SSID will be enterprise, being authenticated using RADIUS. The access point needs to be able to access the internet and all SSIDs should be separated into different VLANs.

Lab Background

Access Points

A wireless access point allows devices that need to use the internet to connect to a WIFI through a beacon request that the AP releases. Every couple of seconds, the AP will advertise the SSID, which devices in range can pick up on and connect. An SSID (Service set identifier) simply exists as the public name of the WIFI being connected to. It serves as a unique identifier which separates itself from other network names. Access points allow the creation of these wireless connections and serve as an entry point into the network. They project the beacon requests which are intercepted by the hosts. When a host connects to the beacon, their traffic is forwarded to the Access point and the Access point will forward all data received from the host to a switch or router, which will forward to an ISP to reach the internet.

Access points often create their own network subnets that are separate from the subnet of a router which is connected to the ISP. An access point connected to an at-home router which leads to the ISP may exist on separate subnets. For instance, the access point may be on the 10.0.0.0/24 subnet, whilst the router would be on the 192.168.1.0/24 subnet. This means that these two subnets are unable to communicate with each other. For this reason, the access point's 10.0.0.0/24 subnet needs to be converted to the 192.168.1.0/24 subnet through NAT. Many access points automatically do this, such as Fortinet's access points, but access points such as Cisco's Aironet do not. Thus, it is recommended to make an intermediary router that NATs all access point traffic to ensure that both networks can communicate.

Multiple SSIDs can run from one Access point. When multiple SSIDs are created, separation of all the data that moves across these networks is necessary because these different networks may have different access levels into the corporate network. Thus, data that runs in between multiple SSIDs are separated by VLANs. However, while it is easy to initially separate data between VLANs once a host's data enters the access point, its necessary to ensure that any VLANs running across any layer 2 or layer 3 devices will maintain this distinction.

Authentication and Encryption

It's very common for attackers to intercept packets going through the network by using tools such as packet tracer. When this does happen, having a secure encryption method such as WPA2 or WPA3 will ensure that only the sender and receiver will have access to the cleartext version of the data. To receive a secure connection, all traffic sent across the wireless network needs to be encrypted. There are numerous encryption methods that can be used but for the purpose of this lab, we used WPA2 and RADIUS. WPA2 uses AES encryption to secure all traffic that goes across the link. Although WPA1 and WPA2 have been cracked, it will throw off most of the inexperienced malicious users but will do little to stop any malicious entity that really wants the data. RADIUS is a method of connection that provides triple A (Authorization, authentication, and accounting) security.

While connecting to the average WIFI connection that runs WPA or WEP, all that is required to connect is the WIFI password. Radius is much different than the WPA series. RADIUS does not provide encryption, which means that an encryption method needs to be used whilst running RADIUS in conjunction. However, RADIUS is used mainly to authorize and authenticate any users connecting to the network. While the average wireless connection only requires a password to access the network, A connection running RADIUS requires both a username and a password to connect, which is created on a RADIUS server. These users are configured with varying permission levels on creation, allowing the network administrator to have more control over who can access what in the network. Multiple users can be created, and all successful or attempted connections are logged and can be viewed by any network administrator that has access to the RADIUS server. Configuring accounting on the RADIUS server will allow any actions done by end users to be logged to the RADIUS server.

When connecting with a host, RADIUS will use either EAP or PEAP authentication to establish a connection. However, in this lab, we only used EAP. To authenticate using EAP, when the RADIUS server receives the initial request from the host to connect, both the server and client will exchange certificates. If the criteria of the client's certificate do not match the server's requirements, then the connection will be denied.

These actions can be separated into the three triple A categories:

Authentication – Users need to verify using their credentials before connecting to the network. Their certificate needs to match the server's certificate.

Authorization – Users are created with different access privileges. To gain more access privileges, a user needs to make a request.

Accounting – Anything the users while attempting to access the internet is logged. This means any websites they visit, the destination of all the user's data, when they logged in and who they logged in as.

This makes RADIUS extremely secure because not only are user credentials stored in an offsite RADIUS server, but any malicious users connected to the network can be easily weeded out because any information about them is logged. If one user is behaving suspiciously, it's easy to see their logs and check if the IP of their company-issued laptop or in-office computer matches with the connected user.

The only issue with using RADIUS to authenticate is the requirement of having a RADIUS server. This requirement presents issues when attempting to run RADIUS on a host machine that is not meant to maintain connection 24/7. When using a service such as free radius on a host, when the host goes down, so will the RADIUS server, meaning the uptime of the wireless connection will depend on the uptime of the host.

However, if a server that can be dedicated to running RADIUS with no downtime is procured, then RADIUS is an extremely powerful tool to ensure full control over users connected to the network.

Verification

From AP to Internet

```
ap#ping 10.0.0.4
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.0.0.4, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/1/2 ms
ap#ping 1.1.1.1
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 1.1.1.1, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 13/219/1011 ms
```

from RADIUS server to internet and AP

```
radius@CCNP-RADIUS:~$ ping 10.0.0.3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=255 time=0.771 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=255 time=5.27 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=255 time=0.767 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=255 time=0.762 ms
^C
--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3038ms
rtt min/avg/max/mdev = 0.762/1.891/5.267/1.948 ms
radius@CCNP-RADIUS:~$ ping 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=53 time=24.6 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=53 time=18.1 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=53 time=18.8 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=53 time=12.2 ms
^C
--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 12.241/18.443/24.613/4.381 ms
radius@CCNP-RADIUS:~$ ping 10.0.0.4
PING 10.0.0.4 (10.0.0.4) 56(84) bytes of data.
64 bytes from 10.0.0.4: icmp_seq=1 ttl=255 time=3.41 ms
64 bytes from 10.0.0.4: icmp_seq=2 ttl=255 time=7.44 ms
64 bytes from 10.0.0.4: icmp_seq=3 ttl=255 time=0.623 ms
64 bytes from 10.0.0.4: icmp_seq=4 ttl=255 time=0.544 ms
^C
--- 10.0.0.4 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.544/3.004/7.443/2.810 ms
radius@CCNP-RADIUS:~$ sudo vim /etc/freeradius/3.0/clients.conf
[sudo] password for radius:
radius@CCNP-RADIUS:~$ sudo vim /etc/freeradius/3.0/users
radius@CCNP-RADIUS:~$
```

Router NAT translations

```
Router#sh ip nat trans
Pro  Inside global         Inside local          Outside local         Outside global
---  192.168.41.160        10.0.0.1              ---                   ---
---  192.168.41.161        10.0.0.67             ---                   ---
---  192.168.41.162        10.0.0.34             ---                   ---
tcp  192.168.41.161:52961  10.0.0.67:52961       35.186.224.36:443     35.186.224.36:443
tcp  192.168.41.161:51301  10.0.0.67:51301       162.159.61.3:443      162.159.61.3:443
tcp  192.168.41.161:51295  10.0.0.67:51295       162.159.61.3:443      162.159.61.3:443
tcp  192.168.41.161:51299  10.0.0.67:51299       162.159.61.3:443      162.159.61.3:443
udp  192.168.41.162:60246  10.0.0.34:60246       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:63617  10.0.0.34:63617       1.1.1.1:53            1.1.1.1:53
tcp  192.168.41.161:54832  10.0.0.67:54832       142.251.211.227:443   142.251.211.227:443
udp  192.168.41.162:64371  10.0.0.34:64371       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:50319  10.0.0.34:50319       23.94.61.242:13255    23.94.61.242:13255
udp  192.168.41.162:62714  10.0.0.34:62714       1.1.1.1:53            1.1.1.1:53
tcp  192.168.41.161:51296  10.0.0.67:51296       162.159.61.3:443      162.159.61.3:443
udp  192.168.41.162:64260  10.0.0.34:64260       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:61524  10.0.0.34:61524       1.1.1.1:53            1.1.1.1:53
tcp  192.168.41.161:54793  10.0.0.67:54793       35.186.224.36:443     35.186.224.36:443
udp  192.168.41.162:61441  10.0.0.34:61441       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:62028  10.0.0.34:62028       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:64993  10.0.0.34:64993       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:58894  10.0.0.34:58894       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:53515  10.0.0.34:53515       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:49887  10.0.0.34:49887       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:50796  10.0.0.34:50796       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:51765  10.0.0.34:51765       1.1.1.1:53            1.1.1.1:53
tcp  192.168.41.161:54790  10.0.0.67:54790       35.186.224.19:443     35.186.224.19:443
udp  192.168.41.162:60066  10.0.0.34:60066       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:59766  10.0.0.34:59766       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:49563  10.0.0.34:49563       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:51738  10.0.0.34:51738       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:57066  10.0.0.34:57066       1.1.1.1:53            1.1.1.1:53
tcp  192.168.41.161:54794  10.0.0.67:54794       162.159.134.234:443   162.159.134.234:443
udp  192.168.41.162:57025  10.0.0.34:57025       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:58791  10.0.0.34:58791       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:53623  10.0.0.34:53623       1.1.1.1:53            1.1.1.1:53
tcp  192.168.41.161:51300  10.0.0.67:51300       172.64.41.3:443       172.64.41.3:443
udp  192.168.41.162:58367  10.0.0.34:58367       1.1.1.1:53            1.1.1.1:53
tcp  192.168.41.161:54820  10.0.0.67:54820       74.125.195.188:5228   74.125.195.188:5228
udp  192.168.41.162:55143  10.0.0.34:55143       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:64620  10.0.0.34:64620       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:62275  10.0.0.34:62275       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:60230  10.0.0.34:60230       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:54268  10.0.0.34:54268       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:57343  10.0.0.34:57343       1.1.1.1:53            1.1.1.1:53
tcp  192.168.41.161:51297  10.0.0.67:51297       162.159.61.3:443      162.159.61.3:443
tcp  192.168.41.161:51298  10.0.0.67:51298       162.159.61.3:443      162.159.61.3:443
tcp  192.168.41.161:54792  10.0.0.67:54792       35.186.224.39:443     35.186.224.39:443
udp  192.168.41.162:51228  10.0.0.34:51228       1.1.1.1:53            1.1.1.1:53
udp  192.168.41.162:61685  10.0.0.34:61685       1.1.1.1:53            1.1.1.1:53
tcp  192.168.41.161:51293  10.0.0.67:51293       162.159.61.3:443      162.159.61.3:443
tcp  192.168.41.161:51302  10.0.0.67:51302       104.154.127.126:4070  104.154.127.126:4070
udp  192.168.41.162:58540  10.0.0.34:58540       1.1.1.1:53            1.1.1.1:53
Total number of translations: 52
```

Verification of connection to radius server

```
(4) eap: Peer sent EAP Response (code 2) ID 6 length 103
(4) eap: Continuing tunnel setup
(4)     [eap] = ok
(4)   } # authorize = ok
(4) Found Auth-Type = eap
(4) # Executing group from file /etc/freeradius/3.0/sites-enabled/default
(4)   authenticate {
(4) eap: Expiring EAP session with state 0x7b4241f878445839
(4) eap: Finished EAP session with state 0x7b4241f878445839
(4) eap: Previous EAP request found for state 0x7b4241f878445839, released from the list
(4) eap: Peer sent packet with method EAP PEAP (25)
(4) eap: Calling submodule eap_peap to process data
(4) eap_peap: (TLS) EAP Peer says that the final record size will be 93 bytes
(4) eap_peap: (TLS) EAP Got all data (93 bytes)
(4) eap_peap: (TLS) Handshake state - Server SSLv3/TLS write server done
(4) eap_peap: (TLS) recv TLS 1.2 Handshake, ClientKeyExchange
(4) eap_peap: (TLS) Handshake state - Server SSLv3/TLS read client key exchange
(4) eap_peap: (TLS) Handshake state - Server SSLv3/TLS read change cipher spec
(4) eap_peap: (TLS) recv TLS 1.2 Handshake, Finished
(4) eap_peap: (TLS) Handshake state - Server SSLv3/TLS read finished
(4) eap_peap: (TLS) send TLS 1.2 ChangeCipherSpec
(4) eap_peap: (TLS) Handshake state - Server SSLv3/TLS write change cipher spec
(4) eap_peap: (TLS) send TLS 1.2 Handshake, Finished
(4) eap_peap: (TLS) Handshake state - Server SSLv3/TLS write finished
(4) eap_peap: (TLS) Handshake state - SSL negotiation finished successfully
(4) eap_peap: (TLS) Connection Established
(4) eap_peap:   TLS-Session-Cipher-Suite = "ECDHE-RSA-AES256-GCM-SHA384"
(4) eap_peap:   TLS-Session-Version = "TLS 1.2"
(4) eap: Sending EAP Request (code 1) ID 7 length 57
(4) eap: EAP session adding &reply:State = 0x7b4241f87f455839
(4)     [eap] = handled
(4)   } # authenticate = handled
(4) Using Post-Auth-Type Challenge
(4) # Executing group from file /etc/freeradius/3.0/sites-enabled/default
(4)   Challenge { ... } # empty sub-section is ignored
(4) session-state: Saving cached attributes
(4)   Framed-MTU = 994
(4)   TLS-Session-Information = "(TLS) recv TLS 1.3 Handshake, ClientHello"
(4)   TLS-Session-Information = "(TLS) send TLS 1.2 Handshake, ServerHello"
(4)   TLS-Session-Information = "(TLS) send TLS 1.2 Handshake, Certificate"
(4)   TLS-Session-Information = "(TLS) send TLS 1.2 Handshake, ServerKeyExchange"
(4)   TLS-Session-Information = "(TLS) send TLS 1.2 Handshake, ServerHelloDone"
(4)   TLS-Session-Information = "(TLS) recv TLS 1.2 Handshake, ClientKeyExchange"
(4)   TLS-Session-Information = "(TLS) recv TLS 1.2 Handshake, Finished"
(4)   TLS-Session-Information = "(TLS) send TLS 1.2 ChangeCipherSpec"
(4)   TLS-Session-Information = "(TLS) send TLS 1.2 Handshake, Finished"
(4)   TLS-Session-Cipher-Suite = "ECDHE-RSA-AES256-GCM-SHA384"
(4)   TLS-Session-Version = "TLS 1.2"
(4) Sent Access-Challenge Id 13 from 10.0.0.10:1812 to 10.0.0.3:1645 length 115
(4)   EAP-Message = 0x0107003919001403030001011603030028a47530921aa9792e917e7ce930851dea765edc0b557aebe83ea2e658895f445a061e57a66d91cd94
(4)   Message-Authenticator = 0x00000000000000000000000000000000
(4)   State = 0x7b4241f87f4558391a9931c657862d34
(4) Finished request
Waking up in 4.9 seconds.
(0) Cleaning up request packet ID 9 with timestamp +5 due to cleanup_delay was reached
```

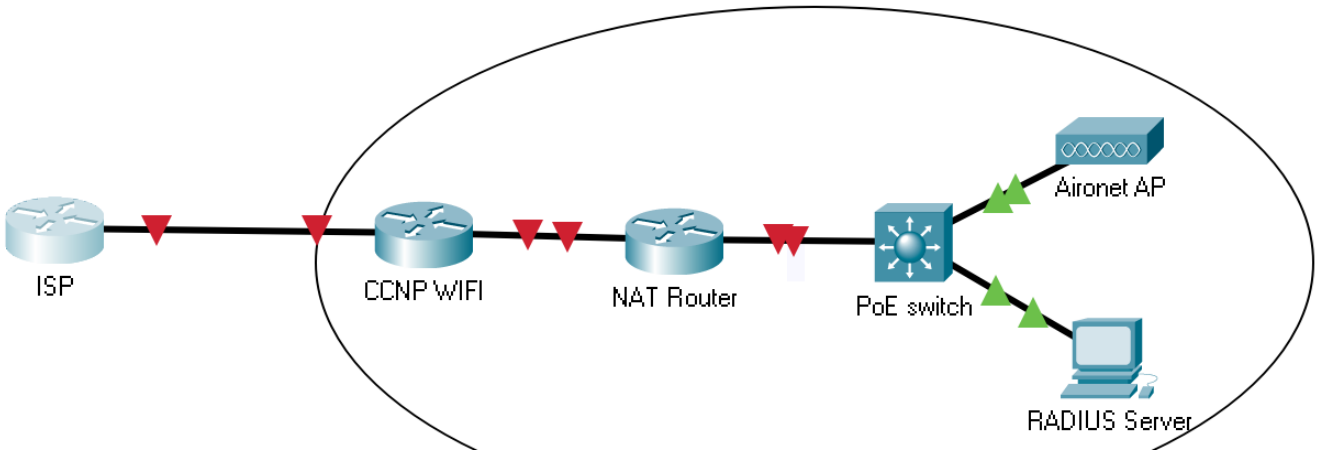
```
(14) eap: Calling submodule eap_peap to process data
(14) eap_peap: (TLS) EAP Done initial handshake
(14) eap_peap: Session established.  Decoding tunneled attributes
(14) eap_peap: PEAP state send tlv success
(14) eap_peap: Received EAP-TLV response
(14) eap_peap: Success
(14) eap: Sending EAP Success (code 3) ID 11 length 4
(14) eap: Freeing handler
(14)     [eap] = ok
(14)   } # authenticate = ok
(14) # Executing section post-auth from file /etc/freeradius/3.0/sites-enabled/default
(14)   post-auth {
(14)     if (session-state:User-Name && reply:User-Name && request:User-Name && (reply:User-Name == request:User-Name)) {
(14)     if (session-state:User-Name && reply:User-Name && request:User-Name && (reply:User-Name == request:User-Name))  -> FALSE
(14)     update {
(14)       &reply::Framed-MTU += &session-state:Framed-MTU[*] -> 994
(14)       &reply::TLS-Session-Information += &session-state:TLS-Session-Information[*] -> '(TLS) recv TLS 1.3 Handshake, ClientHello'
(14)       &reply::TLS-Session-Information += &session-state:TLS-Session-Information[*] -> '(TLS) send TLS 1.2 Handshake, ServerHello'
(14)       &reply::TLS-Session-Information += &session-state:TLS-Session-Information[*] -> '(TLS) send TLS 1.2 Handshake, Certificate'
(14)       &reply::TLS-Session-Information += &session-state:TLS-Session-Information[*] -> '(TLS) send TLS 1.2 Handshake, ServerKeyExchange'
(14)       &reply::TLS-Session-Information += &session-state:TLS-Session-Information[*] -> '(TLS) send TLS 1.2 Handshake, ServerHelloDone'
(14)       &reply::TLS-Session-Information += &session-state:TLS-Session-Information[*] -> '(TLS) recv TLS 1.2 Handshake, ClientKeyExchange'
(14)       &reply::TLS-Session-Information += &session-state:TLS-Session-Information[*] -> '(TLS) recv TLS 1.2 Handshake, Finished'
(14)       &reply::TLS-Session-Information += &session-state:TLS-Session-Information[*] -> '(TLS) send TLS 1.2 ChangeCipherSpec'
(14)       &reply::TLS-Session-Information += &session-state:TLS-Session-Information[*] -> '(TLS) send TLS 1.2 Handshake, Finished'
(14)       &reply::TLS-Session-Cipher-Suite += &session-state:TLS-Session-Cipher-Suite[*] -> 'ECDHE-RSA-AES256-GCM-SHA384'
(14)       &reply::TLS-Session-Version += &session-state:TLS-Session-Version[*] -> 'TLS 1.2'
(14)     } # update = noop
(14)     [exec] = noop
(14)     policy remove_reply_message_if_eap {
(14)       if (&reply:EAP-Message && &reply:Reply-Message) {
(14)       if (&reply:EAP-Message && &reply:Reply-Message)  -> FALSE
(14)       else {
(14)         [noop] = noop
(14)       } # else = noop
(14)     } # policy remove_reply_message_if_eap = noop
(14)     if (EAP-Key-Name && &reply:EAP-Session-Id) {
(14)     if (EAP-Key-Name && &reply:EAP-Session-Id)  -> FALSE
(14)   } # post-auth = noop
(14) Sent Access-Accept Id 23 from 10.0.0.10:1812 to 10.0.0.3:1645 length 175
(14)   MS-MPPE-Recv-Key = 0x6d9e8ee83ada246c259bf45b04970afebf28429c464be06bda59c7750e8ff5bf
(14)   MS-MPPE-Send-Key = 0x0d69300415ec0b721eeeeab44fcb192e35d92f0a195b44b6af15edc270b84292
(14)   EAP-Message = 0x030b0004
(14)   Message-Authenticator = 0x00000000000000000000000000000000
(14)   User-Name = "testing"
(14)   Framed-MTU += 994
(14) Finished request
Waking up in 1.6 seconds.
(5) Cleaning up request packet ID 14 with timestamp +341 due to cleanup_delay was reached
(6) Cleaning up request packet ID 15 with timestamp +341 due to cleanup_delay was reached
(7) Cleaning up request packet ID 16 with timestamp +341 due to cleanup_delay was reached
(8) Cleaning up request packet ID 17 with timestamp +341 due to cleanup_delay was reached
(9) Cleaning up request packet ID 18 with timestamp +341 due to cleanup_delay was reached
Waking up in 3.2 seconds.
```

Lab Summary

First, our access point existed on the K9W8 firmware, which indicates that the access point is in lightweight mode. We needed to download the K9W7 firmware off our tftp server to set the access point to autonomous mode. To set up our WAP to access the internet while maintaining the VLAN information, we created a trunk between the switch and the WAP. Then trunked between the switch and the router which would run NAT. Since our AP existed on a different network than the CCNP network, we had to use the router to NAT between the 10.0.0.0/24 network and the 192.168.40.0/23 network. This allowed our AP to access the CCNP network, which in turn accessed our ISP. After that, we used the Wireless Access Point's GUI to configure the SSIDs. We created a native VLAN and then created 3 vlans for each SSID, which were all allowed on our trunking switch. In order to access the internet from our SSIDs, on the layer 3 switch, we configured DHCP on each VLAN, splitting up the 10.0.0.0/24 network into 4 networks. This allowed all 3 SSIDs to access the internet. To set up the third SSID, which needed RADIUS authentication to connect, we created a separate computer running ubuntu. We downloaded freeradius onto the Linux computer and used it to host our RADIUS server after editing the WAP to be the RADIUS client. To configure this on the Access-point side, we created a remote radius

server, using the RADIUS server's IP address and entered Server properties and SSID manager to allow EAP authentication on SSID 3, setting the RADIUS server as the priority.

Network Diagram



Configurations

Wireless Access Point Configurations for AIR-LAP1042N-a-k9

```
hostname ap
logging rate-limit console 9
enable secret 5 $1$M2CN$frwOUU0L4S3K.zEzGmU.r0

aaa new-model

aaa group server radius rad_eap
server name freeradius

aaa group server radius rad_mac

aaa group server radius rad_acct
```

```
!
aaa group server radius rad_admin

aaa group server tacacs+ tac_admin

aaa group server radius rad_pmip

aaa group server radius dummy

aaa group server radius rad_eap3
server name freeradius

aaa authentication login eap_methods group rad_eap
aaa authentication login mac_methods local
aaa authentication login eap_methods3 group rad_eap3
aaa authorization exec default local
aaa accounting network acct_methods start-stop group rad_acct

aaa session-id common
no ip source-route
no ip cef

dot11 pause-time 100
dot11 syslog

dot11 ssid CCNP_34_111_SSID_1
```

```
   vlan 2
   authentication open
   authentication key-management wpa version 2
   guest-mode
   mbssid guest-mode
   wpa-psk ascii 7 00141215174C04140B701E1D

dot11 ssid CCNP_34_111_SSID_2
   vlan 3
   authentication open
   authentication key-management wpa version 2
   mbssid guest-mode
   wpa-psk ascii 7 06160E325F59060B01464058

dot11 ssid CCNP_34_111_SSID_3
   vlan 4
   authentication open eap eap_methods3
   authentication network-eap eap_methods3
   authentication key-management wpa
   mbssid guest-mode


dot11 wpa handshake timeout 500

no ipv6 cef

username Cisco password 7 01300F175804
```

```
bridge irb

interface Dot11Radio0
no ip address
shutdown

encryption vlan 3 mode ciphers aes-ccm
antenna gain 0
station-role root
bridge-group 1
bridge-group 1 subscriber-loop-control
bridge-group 1 spanning-disabled
bridge-group 1 block-unknown-source
no bridge-group 1 source-learning
no bridge-group 1 unicast-flooding

interface Dot11Radio1
no ip address

encryption vlan 2 mode ciphers aes-ccm

encryption vlan 4 mode ciphers aes-ccm

encryption vlan 3 mode ciphers aes-ccm
```

```
encryption vlan 1 key 2 size 128bit 7
A9E5C82C2DA40E23FF3DBB9FD16E transmit-key
encryption vlan 1 mode wep mandatory

ssid CCNP_34_111_SSID_1

ssid CCNP_34_111_SSID_2

ssid CCNP_34_111_SSID_3

antenna gain 0
peakdetect
dfs band 3 block
mbssid
channel dfs
station-role root

interface Dot11Radio1.1
encapsulation dot1Q 1 native
bridge-group 1
bridge-group 1 subscriber-loop-control
bridge-group 1 spanning-disabled
bridge-group 1 block-unknown-source
no bridge-group 1 source-learning
no bridge-group 1 unicast-flooding

interface Dot11Radio1.2
```

```
encapsulation dot1Q 2
bridge-group 2
bridge-group 2 subscriber-loop-control
bridge-group 2 spanning-disabled
bridge-group 2 block-unknown-source
no bridge-group 2 source-learning
no bridge-group 2 unicast-flooding

interface Dot11Radio1.3
encapsulation dot1Q 3
bridge-group 3
bridge-group 3 subscriber-loop-control
bridge-group 3 spanning-disabled
bridge-group 3 block-unknown-source
no bridge-group 3 source-learning
no bridge-group 3 unicast-flooding

interface Dot11Radio1.4
encapsulation dot1Q 4
bridge-group 4
bridge-group 4 subscriber-loop-control
bridge-group 4 spanning-disabled
bridge-group 4 block-unknown-source
no bridge-group 4 source-learning
no bridge-group 4 unicast-flooding

interface Dot11Radio1.21
```

```
interface Dot11Radio1.31

interface Dot11Radio1.41

interface GigabitEthernet0
no ip address
duplex auto
speed auto

interface GigabitEthernet0.1
encapsulation dot1Q 1 native
bridge-group 1
bridge-group 1 spanning-disabled
no bridge-group 1 source-learning

interface GigabitEthernet0.2
encapsulation dot1Q 2
bridge-group 2
bridge-group 2 spanning-disabled
no bridge-group 2 source-learning

interface GigabitEthernet0.3
encapsulation dot1Q 3
bridge-group 3
bridge-group 3 spanning-disabled
no bridge-group 3 source-learning
```

```
interface GigabitEthernet0.4
encapsulation dot1Q 4
bridge-group 4
bridge-group 4 spanning-disabled
no bridge-group 4 source-learning

interface GigabitEthernet0.21

interface GigabitEthernet0.31

interface GigabitEthernet0.41

interface BVI1
mac-address 44d3.ca5a.86cc
ip address 10.0.0.3 255.255.255.0
ipv6 address dhcp
ipv6 address autoconfig
ipv6 enable

ip default-gateway 192.168.40.1
ip forward-protocol nd
ip http server
no ip http secure-server
ip http help-path
http://www.cisco.com/warp/public/779/smbiz/prodconfig/help/eag
ip radius source-interface BVI1
```

```
radius-server attribute 32 include-in-access-req format %h

radius server freeradius
address ipv4 10.0.0.10 auth-port 1812 acct-port 1813
key 7 0835495D1D100B1043595F

bridge 1 route ip

line con 0
line vty 0 4
transport input all

end
```

SWITCH

IMPORTANT NOTES:

Ap is connected to fa0/33

router is connected to fa0/15

```
hostname Switch1

no aaa new-model
system mtu routing 1500
authentication mac-move permit
ip subnet-zero
ip routing
```

```
no ip domain-lookup
ip dhcp excluded-address 10.0.0.33
ip dhcp excluded-address 10.0.0.3
ip dhcp excluded-address 10.0.0.55
ip dhcp excluded-address 10.0.0.66
ip dhcp excluded-address 10.0.0.99

ip dhcp pool vlan1
   network 10.0.0.32 255.255.255.224
   domain-name calvin.com
   dns-server 1.1.1.1
   default-router 10.0.0.33

ip dhcp pool vlan3
   network 10.0.0.64 255.255.255.224
   dns-server 1.1.1.1
   default-router 10.0.0.65
   domain-name cambl.com

ip dhcp pool vlan4
   network 10.0.0.96 255.255.255.224
   dns-server 1.1.1.1
   default-router 10.0.0.97 255.255.255.224
   domain-name grrrrr.com

crypto pki trustpoint TP-self-signed-1188179200
enrollment selfsigned
```

```
subject-name cn=IOS-Self-Signed-Certificate-1188179200
revocation-check none
rsakeypair TP-self-signed-1188179200

spanning-tree mode pvst
spanning-tree etherchannel guard misconfig
spanning-tree extend system-id

vlan internal allocation policy ascending

interface FastEthernet0/1

interface FastEthernet0/2

interface FastEthernet0/3

interface FastEthernet0/4

interface FastEthernet0/5

interface FastEthernet0/6

interface FastEthernet0/7

interface FastEthernet0/8

interface FastEthernet0/9
```

```
interface FastEthernet0/10

interface FastEthernet0/11

interface FastEthernet0/12

interface FastEthernet0/13

interface FastEthernet0/14

interface FastEthernet0/15
switchport trunk encapsulation dot1q
switchport mode trunk

interface FastEthernet0/16

interface FastEthernet0/17

interface FastEthernet0/18

interface FastEthernet0/19

interface FastEthernet0/20

interface FastEthernet0/21
```

```
interface FastEthernet0/22

interface FastEthernet0/23

interface FastEthernet0/24

interface FastEthernet0/25

interface FastEthernet0/26

interface FastEthernet0/27

interface FastEthernet0/28

interface FastEthernet0/29

interface FastEthernet0/30

interface FastEthernet0/31

interface FastEthernet0/32

interface FastEthernet0/33
switchport trunk encapsulation dot1q
switchport mode trunk

interface FastEthernet0/34
```

```
interface FastEthernet0/35

interface FastEthernet0/36

interface FastEthernet0/37

interface FastEthernet0/38

interface FastEthernet0/39

interface FastEthernet0/40

interface FastEthernet0/41

interface FastEthernet0/42

interface FastEthernet0/43

interface FastEthernet0/44

interface FastEthernet0/45

interface FastEthernet0/46

interface FastEthernet0/47
```

```
interface FastEthernet0/48

interface GigabitEthernet0/1

interface GigabitEthernet0/2

interface GigabitEthernet0/3

interface GigabitEthernet0/4

interface Vlan1
ip address 10.0.0.2 255.255.255.224
no shut

interface Vlan2
ip address 10.0.0.33 255.255.255.224

interface Vlan3
ip address 10.0.0.65 255.255.255.224

interface Vlan4
ip address 10.0.0.97 255.255.255.224

ip default-gateway 10.0.0.1
ip classless
ip route 0.0.0.0 0.0.0.0 10.0.0.1
ip http server
```

```
ip http secure-server

ip sla enable reaction-alerts

line con 0
line vty 0 4
login
line vty 5 15
login
!
end
```

ROUTER CONFIGURATIONS

IMPORTANT NOTES

The sole purpose of this router is to NAT from the network which our AP exists on (10.0.0.0/24) to the CCNP WIFI subnet (192.168.40.0/23)

G0/0/0 connected to wifi switch

G0/0/1 connected to l3 PoE switch

```
hostname Router

address-family ipv4
exit-address-family

address-family ipv6
exit-address-family
```

```
no aaa new-model

subscriber templating
vtp domain cisco
vtp mode transparent
multilink bundle-name authenticated

license udi pid ISR4321/K9 sn FDO21400XZX

spanning-tree extend system-id

redundancy
mode none

vlan internal allocation policy ascending

interface GigabitEthernet0/0/0
ip address dhcp
ip nat outside
negotiation auto

interface GigabitEthernet0/0/1
ip address 10.0.0.1 255.255.255.0
ip nat inside
negotiation auto
```

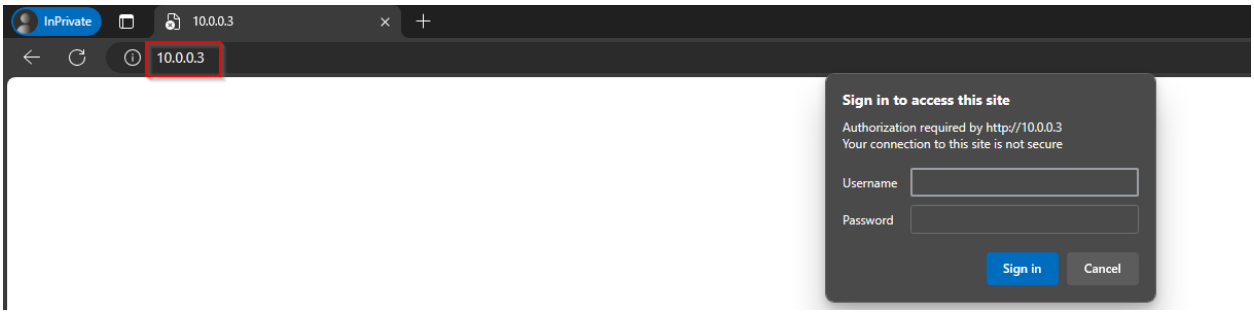
```
ip nat pool pool 192.168.41.160 192.168.41.191 netmask
255.255.255.224

ip nat inside source list 10 pool pool

ip forward-protocol nd

no ip http server

no ip http secure-server

ip tftp source-interface GigabitEthernet0/0/0

access-list 10 permit 10.0.0.0 0.0.0.255

control-plane

ntp server pool.ntp.org
```

Configuration Steps

Configuring Two SSIDs with WPA2

1.

Type in the IP address of the Cisco Wireless Access Point in the search bar. The default IP is 10.0.0.2 if you have not changed it. You should be met with the sign in pop-up. Enter the password (default password and username are both Cisco).



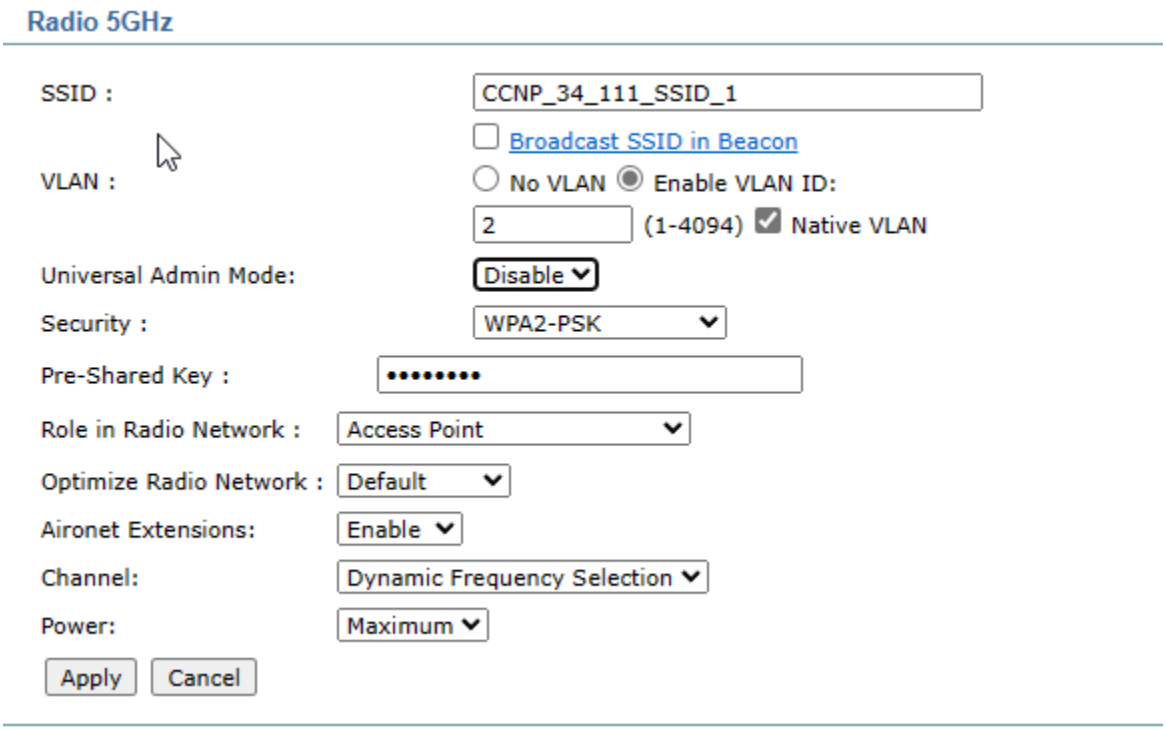
2.

Under the Home tab, enter Easy Setup > Network Configuration.

Create a Host Name, set an IP address if desired and make sure the default gateway is correct.

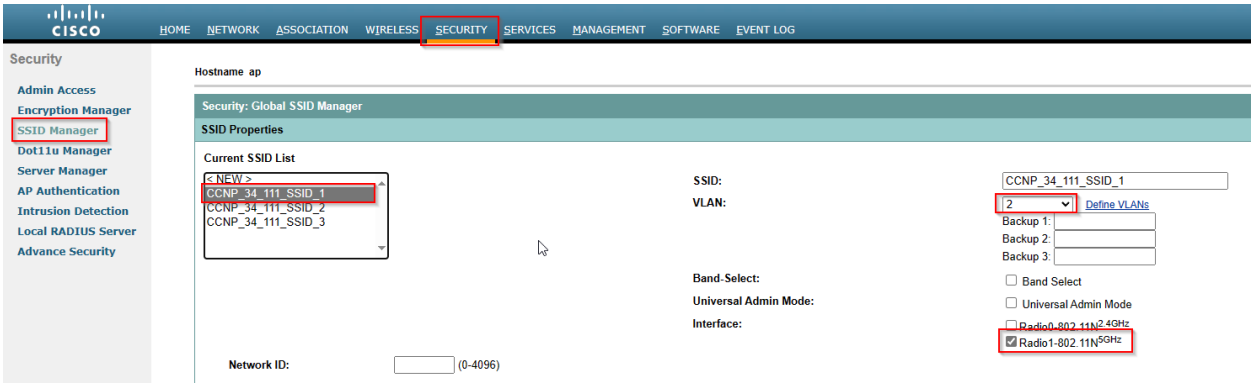
3.

In the same tab, Under Radio 5GHz configure an SSID, set a VLAN number, and set the universal admin mode to disabled and configure the security to WPA2-PSK. Then set a preshared key. Click Apply when done.



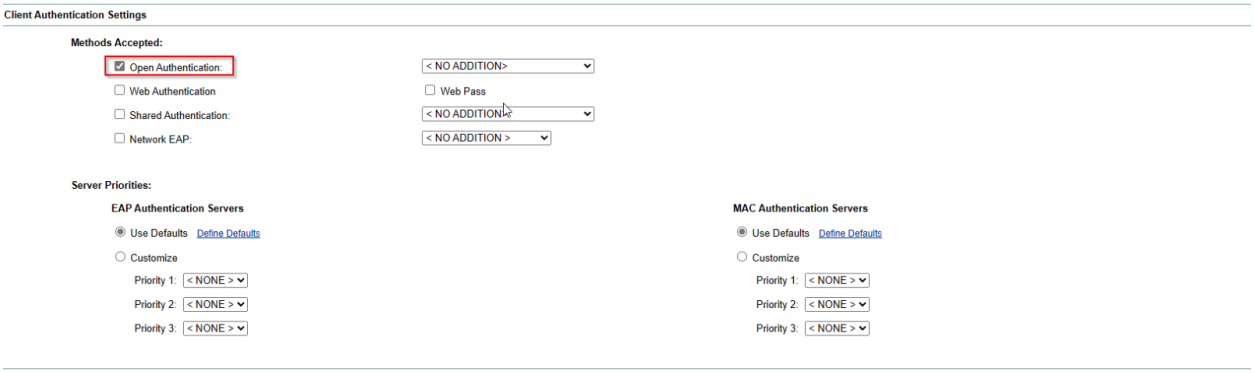
4.

Under the Security Tab, enter SSID Manager. Select the SSID previously configured and make sure the VLAN is set to the correct one and the interface is using 5G.



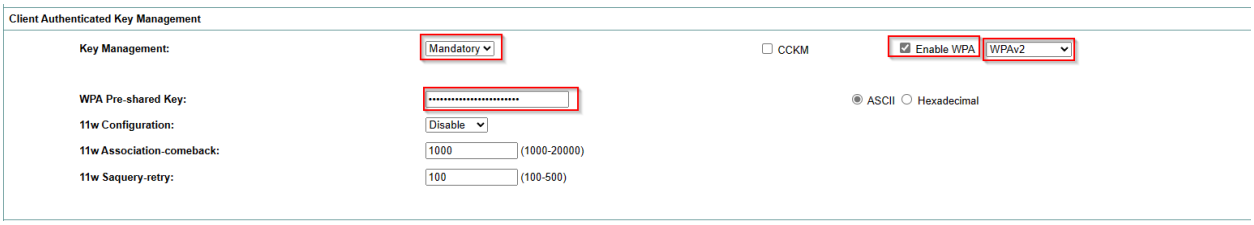
5.

In the same tab, under Client Authentication Settings turn on Open Authentication. This will allow others to connect to the beacon.



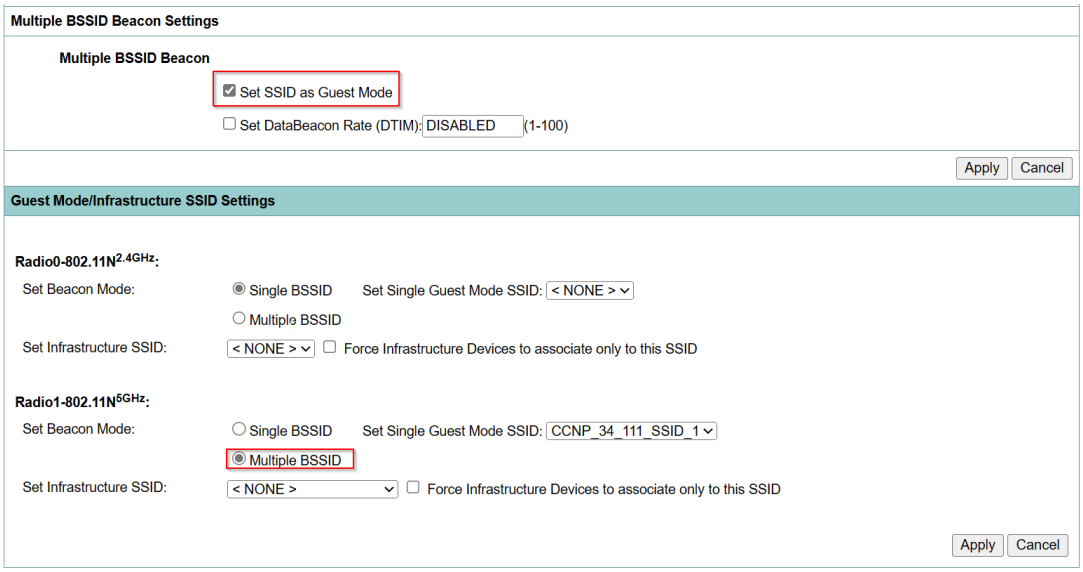
6.

In the same tab, under Client Authenticated Key Management, make sure the Key management is Mandatory and WPA is enabled using WPAv2. The Pre-shared Key should be the one created in step 3.



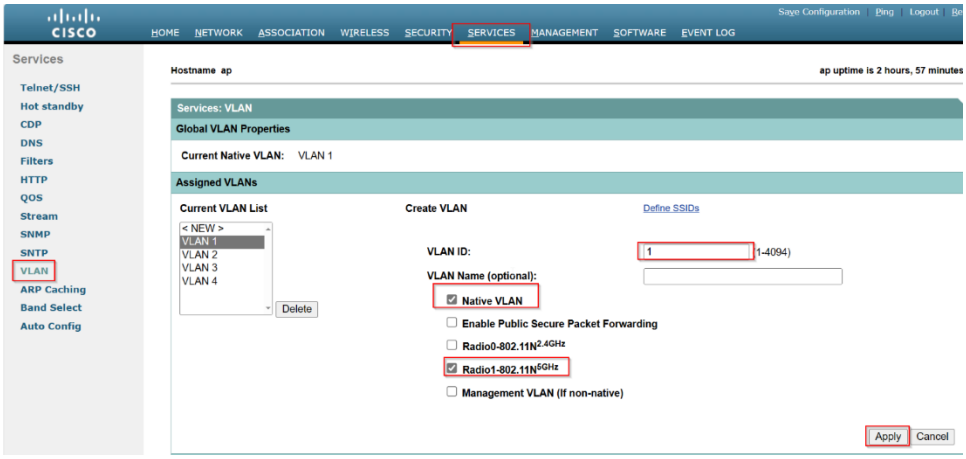
7.

Under Multiple BSSID Beacon Settings, set the SSID as Guest Mode. Then Under Guest Mode/Infrastructure SSID Settings, set the Beacon Mode to Multiple BSSID. This will allow the AP to create multiple SSIDs and broadcast them all.



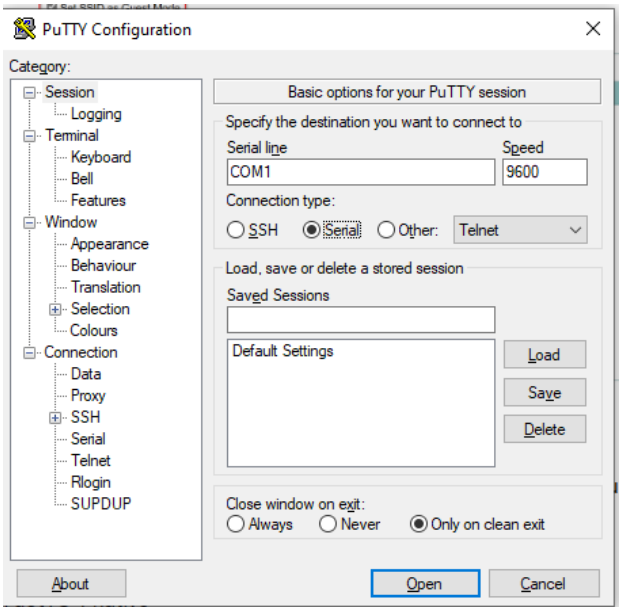
8.

Under the Services tab, enter the VLAN service. Then Create a Native vlan. Bind the native vlan to a VLAN of your choice and then make sure it is set to 5G. Then Click Apply.



9.

Enter the Switch connected to the Access Point with the default password and username being Cisco.



10.

In each of the Dot11Radio1 interfaces created in the Gui, Enter the Native vlan Interface created in step 8 and bind it to the Native VLAN.

interface Dot11Radio1.(vlanNumber)

for instance, since i created VLAN 1 as the native VLAN, i would use

interface Dot11Radio1.1

Then run

encapsulation dot1Q 1 native to bind it to the native VLAN.

11.

Then enter the previously configured VLAN the SSID uses and enter the interface using

interface Dot11Radio(vlan_number)

Then bind it to a normal VLAN that is **not** native using

encapsulation dot1q 1

12. Repeat Steps 1 to 11 to create the second SSID. The second SSID should be identical to the first ID in configuration except for the VLAN bindings and password.

Configuring RADIUS

1. Open linux terminal

2. Enter sudo su mode, which allows you to execute commands that require administrative access. Then download freeradius using the apt-get install freeradius command.

```
radius@CCNP-RADIUS:~$ sudo su
root@CCNP-RADIUS:/home/radius# apt-get freeradius
E: Invalid operation freeradius
root@CCNP-RADIUS:/home/radius# apt-get install freeradius
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
freeradius is already the newest version (3.0.26~dfsg~git20220223.1.00ed0241fa-0ubuntu3.2).
The following packages were automatically installed and are no longer required:
  libwpe-1.0-1 libwpebackend-fdo-1.0-1
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
root@CCNP-RADIUS:/home/radius#
```

3. run a "vim" command, on /etc/freeradius/3.0/clients.conf. The command "vim" allows you to directly edit this config file.

```
root@CCNP-RADIUS:/home/radius# vim /etc/freeradius/3.0/clients.conf
```

4. You should see a similar screen to this. If not, scroll up and down using the arrow keys to get to this area.

```
#  i.e. The entry from the smallest possible network.
#
#client private-network-1 {
#       ipaddr          = 192.0.2.0/24
#       secret          = testing123-1
#}

#client private-network-2 {
#       ipaddr          = 198.51.100.0/24
#       secret          = testing123-2
#}

######################################################################
#
#  Per-socket client lists.  The configuration entries are exactly
#  the same as above, but they are nested inside of a section.
#
#  You can have as many per-socket client lists as you have "listen"
#  sections, or you can re-use a list among multiple "listen" sections.
#
#  Un-comment this section, and edit a "listen" section to add:
#  "clients = per_socket_clients".  That IP address/port combination
#  will then accept ONLY the clients listed in this section.
#
#  There are additional considerations when using clients from SQL.
#
#  A client can be link to a virtual server via modules such as SQL.
#  This link is done via the following process:
#
#  If there is no listener in a virtual server, SQL clients are added
#  to the global list for that virtual server.
#
#  If there is a listener, and the first listener does not have a
#  "clients=..." configuration item, SQL clients are added to the
#  global list.
#
#  If there is a listener, and the first one does have a "clients=..."
#  configuration item, SQL clients are added to that list.  The client
#  { ...} ` configured in that list are also added for that listener.
#
#  The only issue is if you have multiple listeners in a virtual
#  server, each with a different client list, then the SQL clients are
#  added only to the first listener.
#
#clients per_socket_clients {
#       client socket_client {
#               ipaddr = 192.0.2.4
#               secret = testing123
#       }
#}
client accesspoint {
        secret = testing123
        ipaddr = 10.0.0.3
}
"/etc/freeradius/3.0/clients.conf" 292L, 8343B
```

Directory: /etc/freeradius/3.0/clients.conf

Access by using “sudo vim /etc/freeradius/3.0/clients.conf”

This is where you create a RADIUS client which should be the IP address of the AP

Format:

client [name] {

secret = [password]

ipaddr = [ip-address]

}

```
##########################################################################
#
#  Per-socket client lists.  The configuration entries are exactly
#  the same as above, but they are nested inside of a section.
#
#  You can have as many per-socket client lists as you have "listen"
#  sections, or you can re-use a list among multiple "listen" sections.
#
#  Un-comment this section, and edit a "listen" section to add:
#  "clients = per_socket_clients".  That IP address/port combination
#  will then accept ONLY the clients listed in this section.
#
#  There are additional considerations when using clients from SQL.
#
#  A client can be link to a virtual server via modules such as SQL.
#  This link is done via the following process:
#
#  If there is no listener in a virtual server, SQL clients are added
#  to the global list for that virtual server.
#
#  If there is a listener, and the first listener does not have a
#  "clients=..." configuration item, SQL clients are added to the
#  global list.
#
#  If there is a listener, and the first one does have a "clients=..."
#  configuration item, SQL clients are added to that list.  The client
#  { ...} ` configured in that list are also added for that listener.
#
#  The only issue is if you have multiple listeners in a virtual
#  server, each with a different client list, then the SQL clients are
#  added only to the first listener.
#
#clients per_socket_clients {
#       client socket_client {
#               ipaddr = 192.0.2.4
#               secret = testing123
#       }
#}
client accesspoint {
        secret = testing123
        ipaddr = 10.0.0.3
}
```

Directory: /etc/freeradius/3.0/users

access by using sudo vim /etc/freeradius/3.0/users

This is where you will create or delete users to access the SSID.

Format:

[username] Cleartext-Password := “[password]”

```
testing Cleartext-Password := "password"
cambell Cleartext-Password := "cambell"
#
#
#       Configuration file for the rlm_files module.
#       Please see rlm_files(5) manpage for more information.
#
#       This file contains authentication security and configuration
#       information for each user.  Accounting requests are NOT processed
#       through this file.  Instead, see 'accounting', in this directory.
#
#       The first field is the user's name and can be up to
#       253 characters in length.  This is followed (on the same line) with
#       the list of authentication requirements for that user.  This can
#       include password, comm server name, comm server port number, protocol
#       type (perhaps set by the "hints" file), and huntgroup name (set by
#       the "huntgroups" file).
#
#       If you are not sure why a particular reply is being sent by the
#       server, then run the server in debugging mode (radiusd -X), and
#       you will see which entries in this file are matched.
#
#       When an authentication request is received from the comm server,
#       these values are tested. Only the first match is used unless the
#       "Fall-Through" variable is set to "Yes".
#
#       A special user named "DEFAULT" matches on all usernames.
#       You can have several DEFAULT entries. All entries are processed
#       in the order they appear in this file. The first entry that
#       matches the login-request will stop processing unless you use
#       the Fall-Through variable.
#
#       Indented (with the tab character) lines following the first
#       line indicate the configuration values to be passed back to
#       the comm server to allow the initiation of a user session.
#       This can include things like the PPP configuration values
#       or the host to log the user onto.
#
#       You can include another `users' file with `$INCLUDE users.other'

#
#       For a list of RADIUS attributes, and links to their definitions,
#       see: http://www.freeradius.org/rfc/attributes.html
```

5. Press I to enter insert mode which allows to edit the document. Create a client by using the client access point { secret = password, ipaddr = x.x.x.x } command. This will establish your access point as the client of this radius server. A similar command needs to be established on the wireless access point to establish this pc as the server. Enter a password that will remain constant between your AP and the server and the IP address of your wireless access point.

```
client accesspoint {
        secret = testing123
        ipaddr = 10.0.0.3
```

6. press escape to exit insert mode and type in ":wq", which specifies to exit vim mode and save all edits. Keep in mind that typing in ":q" will exit without saving in case of any misconfiguration.

```
:wq
```

7. Next, run a vim, specifying the same directory, but entering the "users" config file.

```
root@CCNP-RADIUS:/home/radius# vim /etc/freeradius/3.0/users
```

8. You should see a screen similar to this. Note the INSERT in all caps at the bottom of the screen, which indicates you are in insert mode.

```
testing Cleartext-Password := "password"
#
#
#       Configuration file for the rlm_files module.
#       Please see rlm_files(5) manpage for more information.
#
#       This file contains authentication security and configuration
#       information for each user.  Accounting requests are NOT processed
#       through this file.  Instead, see 'accounting', in this directory.
#
#       The first field is the user's name and can be up to
#       253 characters in length.  This is followed (on the same line) wit
#       the list of authentication requirements for that user.  This can
#       include password, comm server name, comm server port number, proto
#       type (perhaps set by the "hints" file), and huntgroup name (set by
#       the "huntgroups" file).
#
#       If you are not sure why a particular reply is being sent by the
#       server, then run the server in debugging mode (radiusd -X), and
#       you will see which entries in this file are matched.
#
#       When an authentication request is received from the comm server,
#       these values are tested. Only the first match is used unless the
#       "Fall-Through" variable is set to "Yes".
#
#       A special user named "DEFAULT" matches on all usernames.
#       You can have several DEFAULT entries. All entries are processed
#       in the order they appear in this file. The first entry that
#       matches the login-request will stop processing unless you use
#       the Fall-Through variable.
#
#       Indented (with the tab character) lines following the first
#       line indicate the configuration values to be passed back to
#       the comm server to allow the initiation of a user session.
#       This can include things like the PPP configuration values
#       or the host to log the user onto.
#
#       You can include another `users' file with `$INCLUDE users.other'
#
#       For a list of RADIUS attributes, and links to their definitions,
#       see: http://www.freeradius.org/rfc/attributes.html
#
#       Entries below this point are examples included in the server for
#       educational purposes. They may be deleted from the deployed
#       configuration without impacting the operation of the server.
#
#
#
# Deny access for a specific user.  Note that this entry MUST
# be before any other 'Auth-Type' attribute which results in the user
# being authenticated.
#
-- INSERT --
```

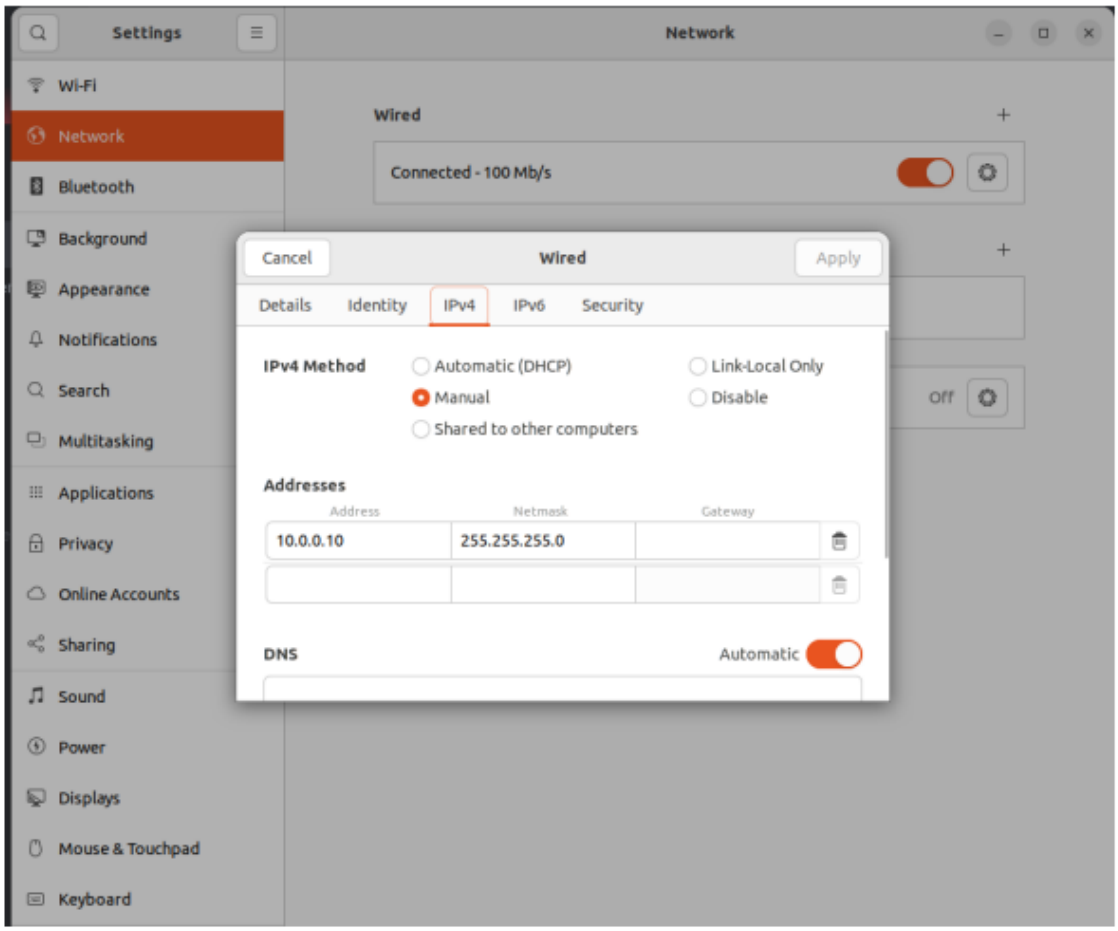
9. Notice the pre-configured user that is already at the top of the file. If there is no pre-configured user, create one by typing (User) Cleartext-Password := "(password)"

```
testing Cleartext-Password := "password"
```

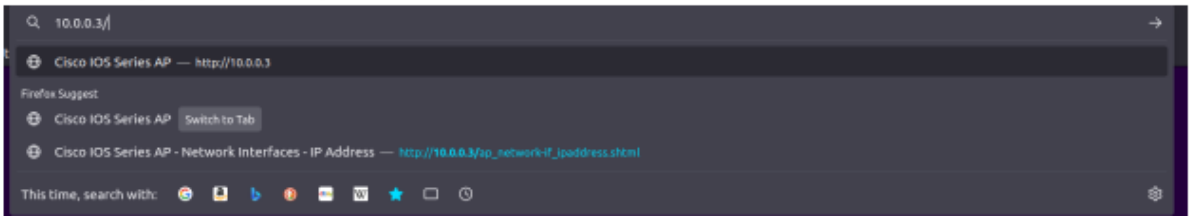
10. Exit insert mode and type in ":wq" again to exit and save.

```
:wq
```

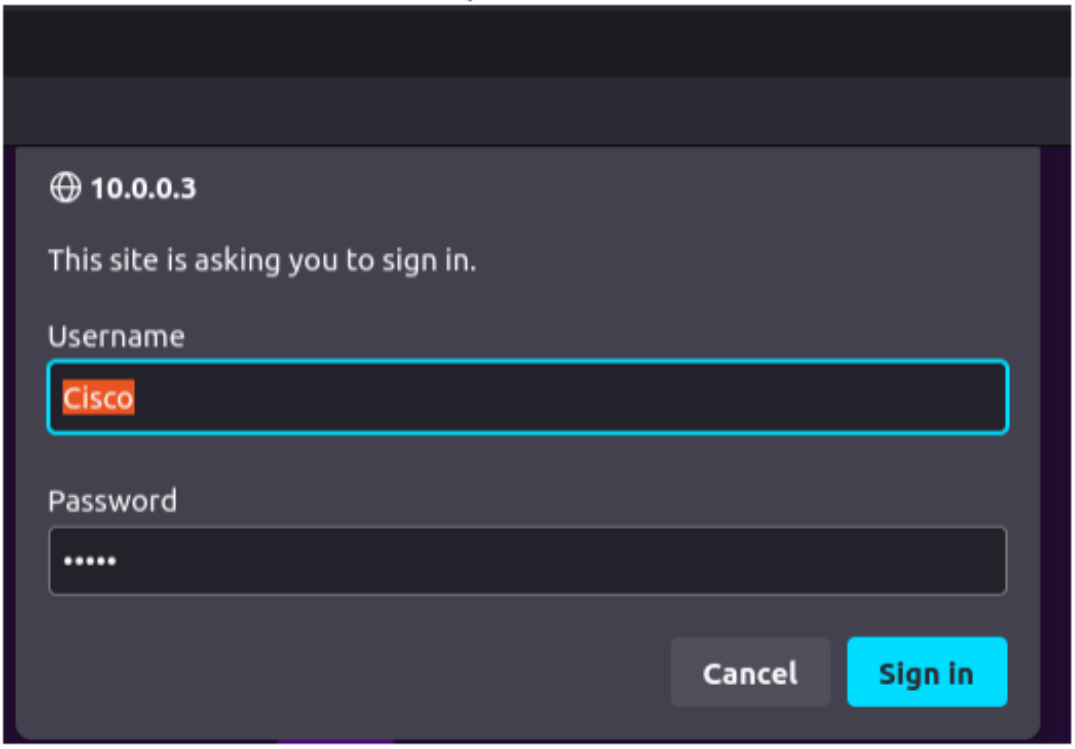
11. Edit your ETHERNET connection and create a static ip address. This IP will be the ip address of your RADIUS server because it is running on your own PC. Remember this IP.



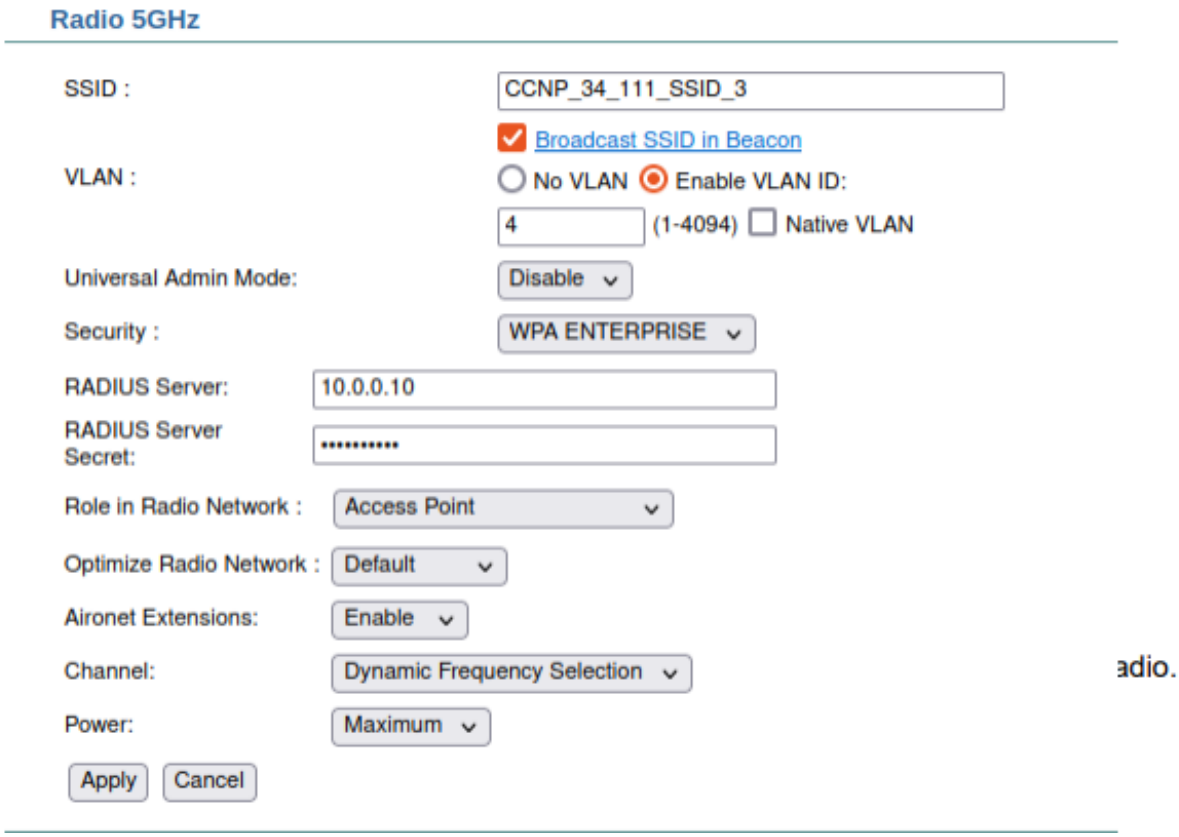
12. enter a new tab and type in the IP address of the wireless AP into the search bar. The default IP is 10.0.0.2 if you have not changed your IP. (though note that if there are multiple of these APs on the same LAN and with the default IP, it will result in overlapping IP addresses).



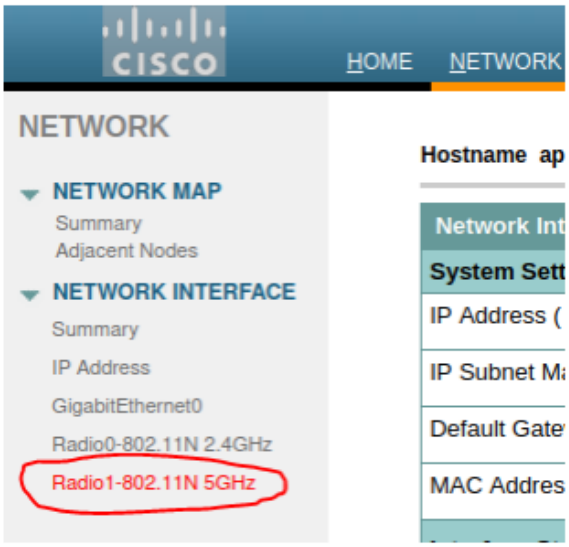
13. You should get this prompt, type in your username and password and log in. The default username is Cisco and the default password is Cisco



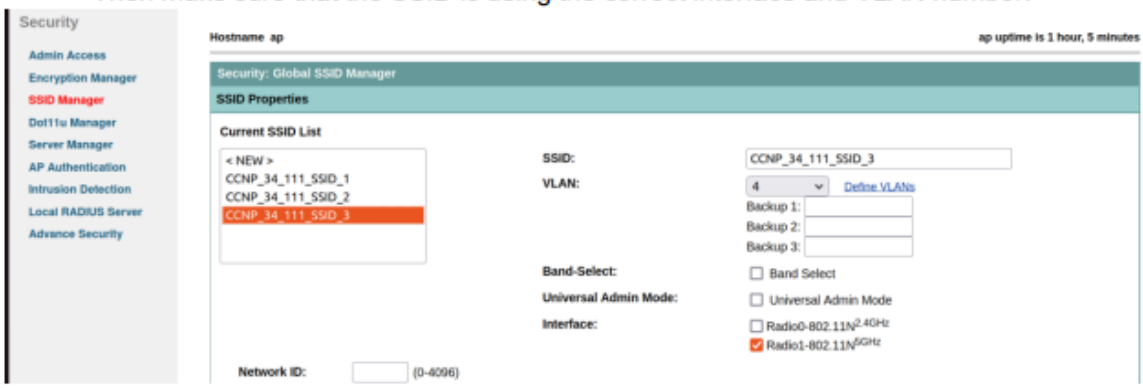
14. Create an SSID and assign it a VLAN. Point it towards the RADIUS server's ip address and type in the RADIUS server's Secret which was configured in step 5.



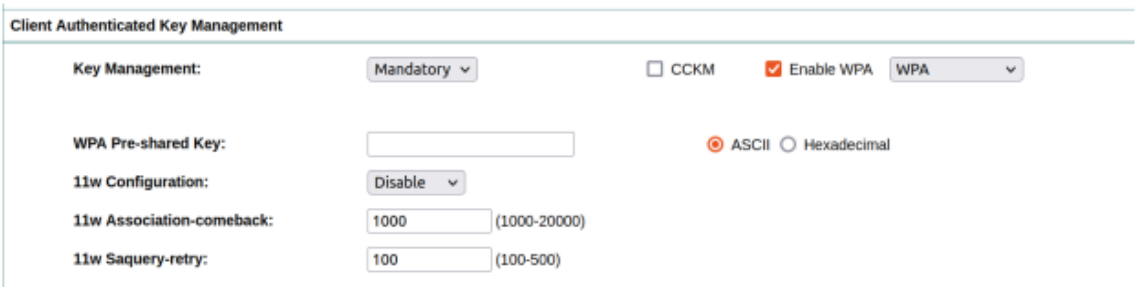
adio.



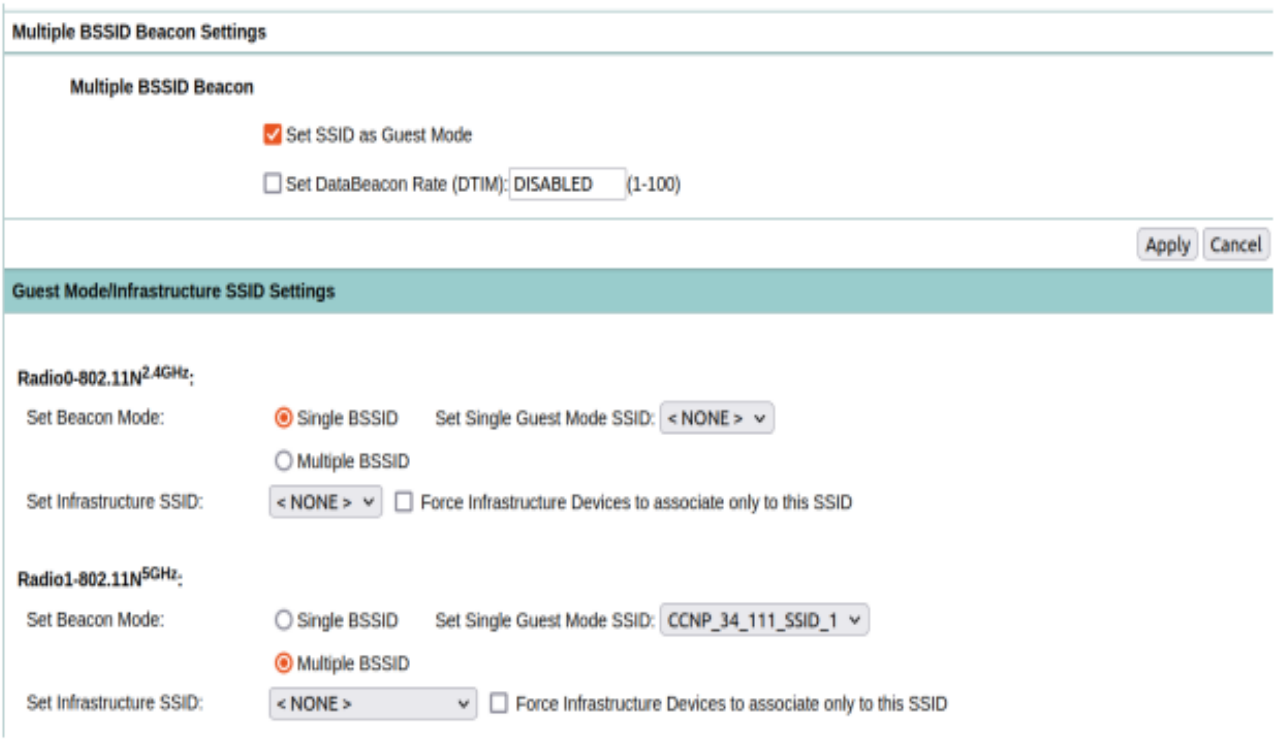
16. Enter Security > SSID Manager and specify that you are configuring the RADIUS SSID. Then make sure that the SSID is using the correct interface and VLAN number.



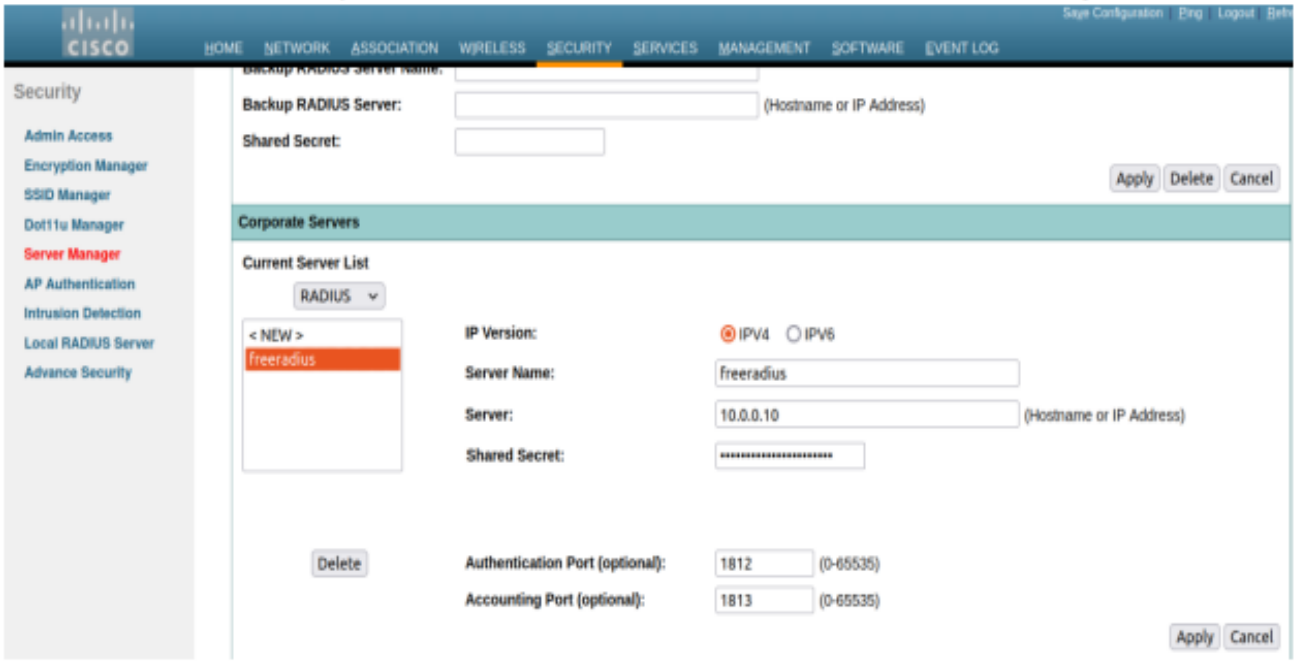
16.5. Scroll down to Client Authentication Key management and allow WPA.



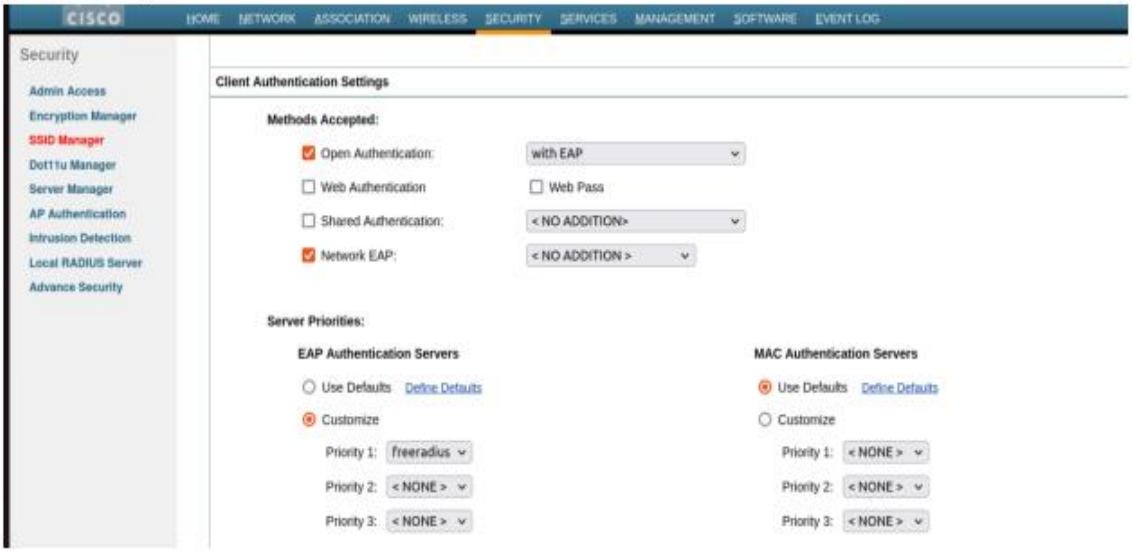
17. Under Multiple BSSID Beacon settings, set the SSID to be in guest mode. Then under Guest/Infrastructure Settings, set the beacon to multiple BSSID mode. This will allow your Beacon to be broadcasted.



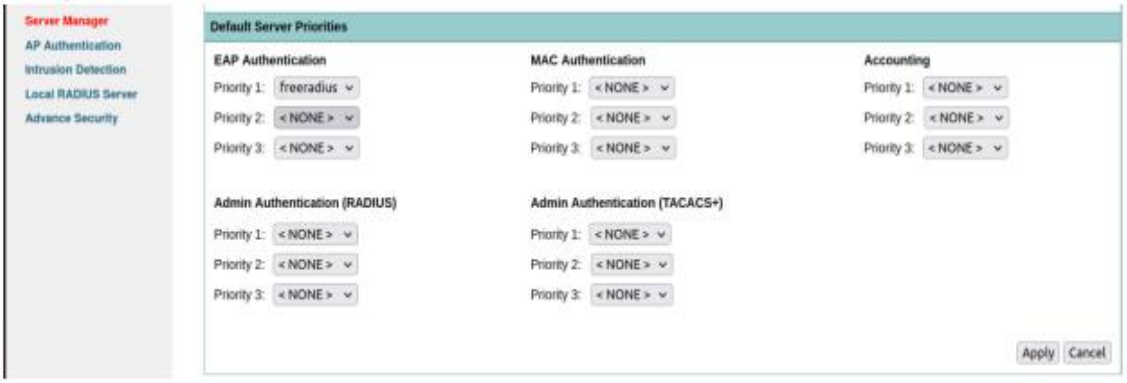
18. Under the Server Managers tab, create a new server, specify the server name, IP, and the shared secret. Specify ports 1812 and 1813 for authentication and accounting.



19. Go back into SSID Manager and enable Open Authentication with EAP and Network EAP. Then specify the server priorities to use the radius server you configured in the previous step.



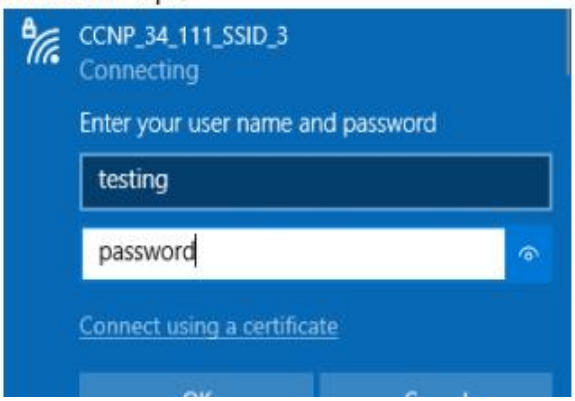
20. After enabling EAP, go back to Server Manager and set priority 1 to use your previously configured radius server.



21. Go back into your radius server's linux config and run a freeradius -X with root privileges.

```
radius@CCNP-RADIUS:~$ sudo freeradius -X
```

22. connect to the RADIUS wifi using the previously configured test user's credentials we created in step 9



Linux Commands

Sudo

Adding sudo in front of any command will extend that command to have root privileges

sudo service freeradius stop

stops the freeradius instance

sudo freeradius –X

starts the freeradius instance

vim

edits a text/config file

Problems

Local RADIUS server on cisco WAP not working

When a local RADIUS server is configured on the WAP, the WAP cannot make its own encryption. With how little documentation the WAP has, it was very difficult for us to find this out and ended up taking a couple days before figuring out what went wrong. The local radius server also required us to add a certificate because windows doesn't trust the connection to the WIFI.

Using Freeradius on WSL

While it is possible to download FreeRADIUS on WSL, the RADIUS server on WSL was still unable to access outside of the host machine. Thus, WSL was unable to ping the cisco AP without some sort of port forwarding which overcomplicated the lab greatly.

Trying to download RADIUS on a VM

Before attempting WSL, we attempted to create our own RADIUS server on a VM. However, we had a similar issue with using WSL, where it was difficult to allow the VM server to ping outside of the host machine.

Issues with Cisco WAP GUI

Each time we made a change to the GUI, it would require us to refresh the page which often loads very slowly and could take up to a minute to load fully. Even after this, our GUI would often crash for tens of minutes at a time for no apparent reason, which it will load back up on its own after enough time passes. Sometimes the GUI would have completely different configurations than the CLI for example, on the GUI it forces you to use CCMP + Tkip for encryption, while in the CLI, you can use CCMP on its own.

Issues with Physical topology

Since the Rack me and my partner was connected to also contained every access point's connection to the internet, we often had issues with duplicate IP addresses if another group's NAT configuration was incorrect. It often also slowed down our progress because it was difficult to troubleshoot our issues because we couldn't directly access the internet switch out of fear of shutting it down which would take out all other group's internet as well.

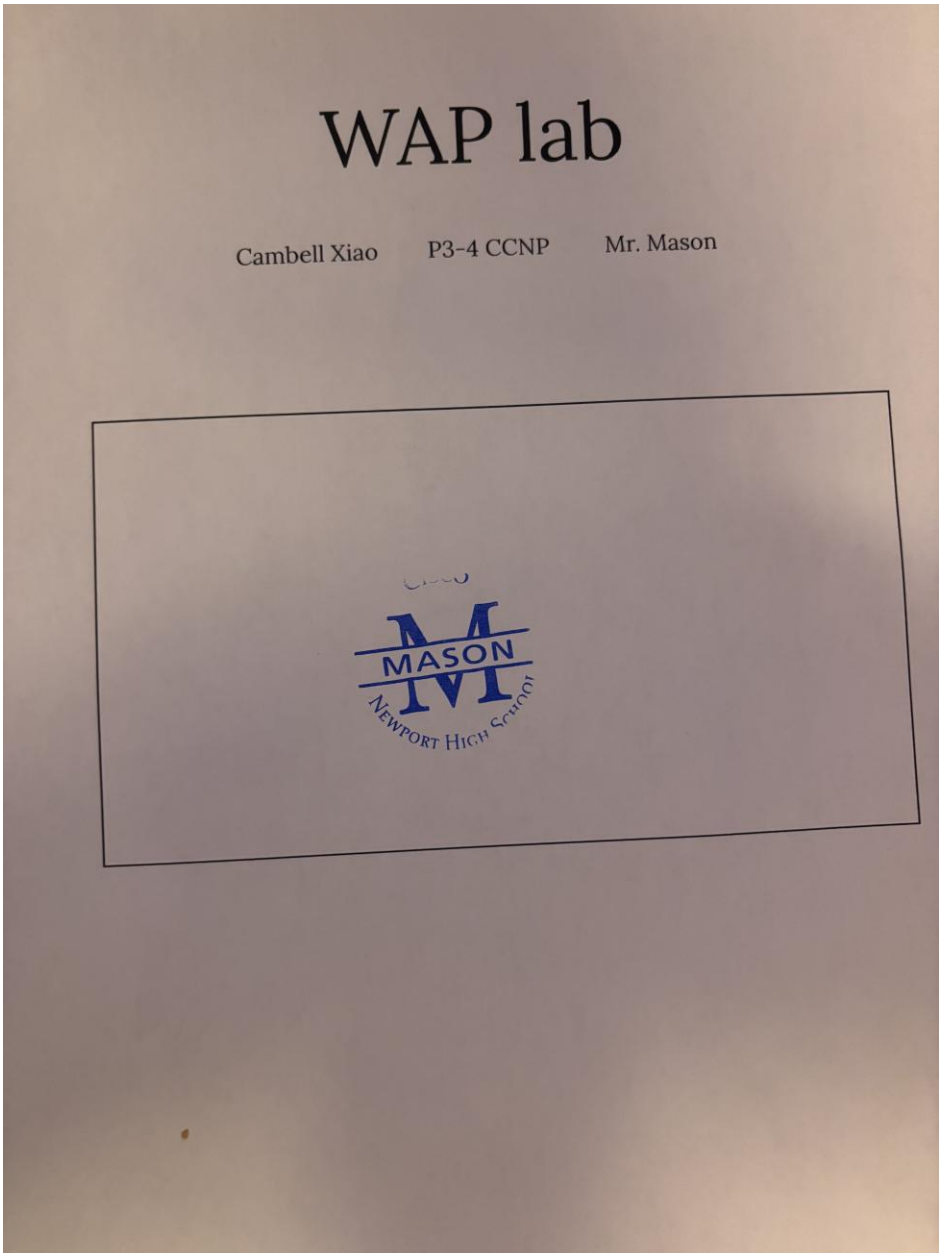
Finding documentation for this specific WAP model.

There was very little documentation on the internet regarding this WAP model. While there was enough documentation to get the first two SSIDs up, there was very little documentation to configure a local radius server because it was not recommended to do and thus nobody attempted it. We couldn't find evidence of anyone succeeding in setting up a local radius server on this WAP.

Conclusion

While we did run into many issues regarding the confusing GUI of the Cisco AP, if we can look past the many issues the access point has regarding configuration in the GUI, it is still a solid choice. Configuring SSIDs is very simple, however getting the networks protected through triple A and encryption becomes much more complicated. For instance, our Cisco Aironet only encrypted up to WPA2, which was still outdated. Overall, the most challenging process of this lab was configuring RADIUS, which usually shouldn't be an issue provided a local RADIUS server on the access point works or if we had a dedicated server to run RADIUS on. Creating SSIDs to access the network requires strong foundational networking knowledge regarding IP routing, namely VLANs, switchports, and network addressing and wouldn't prove challenging to an experienced network engineer.

SIGN OFF


WAP lab
Cambell Xiao
P3-4 CCNP
Mr. Mason
MASON
Newport High School